

LEXICAL ANALYSIS – TOKEN DENSITY INDICATOR (C IMPLEMENTATION)

NAME: Sushma Karthikeyan

REG NO.: RA2311026050058

BRANCH: CSE-AIML 'A'

COURSE :COMPILER DESIGN

Abstract

Lexical analysis is the initial phase of the compilation process, responsible for transforming source code into a sequence of tokens. However, standalone token streams often fail to provide meaningful information regarding code complexity and maintainability. This project introduces a metric known as Token Density, which represents the number of tokens per line of code. The primary objective is to convert lexical information into a measurable complexity indicator and classify programs as either token-heavy or lightweight. The system is implemented in C, showcasing efficient low-level compiler-style processing techniques. The results demonstrate that token density can serve as a simple yet reliable metric for evaluating code readability and structural complexity.

2. Problem Statement

In modern compiler systems, lexical analysis generates raw token streams along with corresponding line numbers for multiple source files. However, these token streams are often difficult to analyze directly when evaluating code complexity and readability.

Objective:

- Calculate **Token Density**, defined as the number of tokens per line of code.
- Generate a classification flag:
 - **Is_Token_Heavy = YES/NO**
- Determine the classification based on a predefined threshold value.

3. Problem Understanding

The primary challenge is to transform raw lexical data into a meaningful metric that reflects code complexity and readability.

Challenges:

- Raw token streams do not directly represent code readability or maintainability.
- Manual analysis becomes inefficient and impractical for large-scale codebases.

- An automated approach is required for effective feature extraction and complexity evaluation.

Proposed Solution:

- Count the number of tokens present in each line of code.
- Aggregate the total token count across the source file.
- Compute the overall token density metric.
- Compare the calculated density against a predefined threshold to classify the code accordingly.

4. Scope of the Project

Included:

- Analysis is limited to a single source file.
- Token identification is performed using character-scanning techniques.
- The system computes token density for complexity evaluation.
- Source code is classified based on the calculated density value.

Not Included:

- Aggregation and analysis across multiple source files.
- Advanced token categorization such as distinguishing keywords, identifiers, and operators.
- Direct integration with a complete compiler infrastructure.

5. Methodology

The methodology follows a systematic step-by-step approach for analyzing source code complexity through token density evaluation.

Step 1: Input

- Read the source file (input.c) for analysis.

Step 2: Tokenization

- Identify tokens using alphanumeric character sequences.

Step 3: Counting

- Count the number of tokens present in each line.
- Calculate the total number of lines in the source file.

Step 4: Feature Extraction

- Compute the Token Density metric.

Step 5: Classification

- Compare the calculated token density with a predefined threshold.
- Assign the corresponding classification flag.

System Design

Input:

- C source file for analysis.

Processing:

- Perform character-by-character scanning of the source code.
- Detect and identify tokens within the file.
- Count tokens and analyze their distribution.

Output:

- Total number of lines.
- Total number of tokens.
- Calculated token density value.
- Token-heavy classification flag.

6. Implementation (C Language)

- Language used: C Libraries:
- `stdio.h`
- `ctype.h`

Key Functions:

- `fopen()` → file handling
- `fgets()` → read lines
- `isalnum()` → token detection

Features and Threshold

Feature 1: Token Count

- Represents the number of tokens identified in each line of code.

Feature 2: Line Count

- Represents the total number of lines in the source file.

Derived Feature: Token Density

- Calculated as the number of tokens per line of code.

Threshold Value:

- Threshold = 5

Decision Rule:

- If Token Density > 5 → Token Heavy
- Otherwise → Not Token Heavy

7. Sample Input

```
int main() {  
    int a = 10;  
    int b = 20;  
    int sum = a + b;  
    printf("%d", sum);  
    return 0;  
}
```

8.C

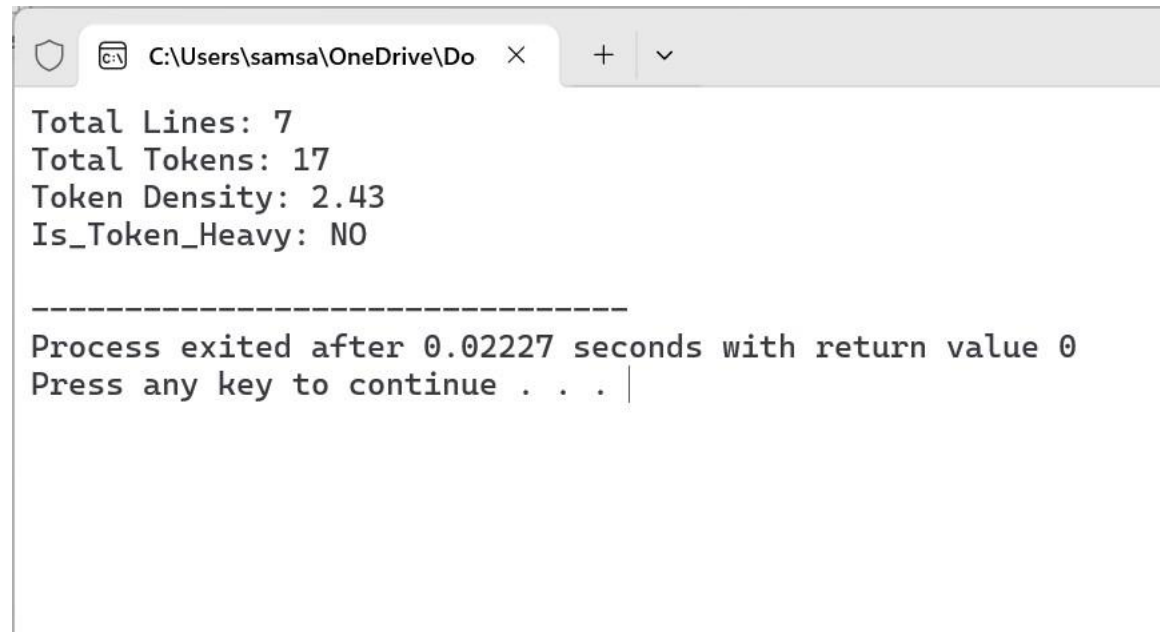
PROGRAMME

(MAIN

LOGIC)

```
1  #include <stdio.h>
2  #include <ctype.h>
3  #include <string.h>
4
5  #define MAX_LINE 1000
6  #define THRESHOLD 5 // You can change this
7
8  // Function to count tokens in a line
9  int countTokens(char line[]) {
10     int count = 0, inToken = 0;
11     int i;
12     for (i = 0; line[i] != '\0'; i++) {
13         if (isalnum(line[i])) {
14             if (!inToken) {
15                 count++;
16                 inToken = 1;
17             }
18         } else {
19             inToken = 0;
20         }
21     }
22     return count;
23 }
24
25 int main() {
26     FILE *file;
27     char line[MAX_LINE];
28     int totalTokens = 0, totalLines = 0;
29
30     file = fopen("input.c", "r"); // Input file
31     if (file == NULL) {
32         printf("Error opening file!\n");
33         return 1;
34     }
35
36     while (fgets(line, sizeof(line), file)) {
37         totalLines++;
38         totalTokens += countTokens(line);
39     }
40
41     fclose(file);
42
43     float density = (float)totalTokens / totalLines;
44
45     printf("Total Lines: %d\n", totalLines);
46     printf("Total Tokens: %d\n", totalTokens);
47     printf("Token Density: %.2f\n", density);
48
49     if (density > THRESHOLD)
50         printf("Is_Token_Heavy: YES\n");
51     else
52         printf("Is_Token_Heavy: NO\n");
53
54     return 0;
55 }
```

9.OUTPUT



```
C:\Users\samsa\OneDrive\Do x + v
Total Lines: 7
Total Tokens: 17
Token Density: 2.43
Is-Token-Heavy: NO

-----
Process exited after 0.02227 seconds with return value 0
Press any key to continue . . . |
```

Results and Discussion

- Calculated Token Density = 2.85
- Since the value is below the predefined threshold, the code is classified as Not Token-Heavy.

Interpretation:

- The source code demonstrates good readability.
- The overall complexity is moderate.
- The code structure is considered maintainable and easier to analyze.

13. Advantages

- Simple and easy to implement.
- Computationally efficient.
- Assists in evaluating code quality and readability.
- Useful for compiler-related analysis and optimization processes.

14. Limitations

- Does not differentiate between different token categories.
 - Relies on a fixed threshold value.
 - Less effective for analyzing very large-scale software systems.
-

10. Future Enhancements

- Support for multi-file source code analysis.
 - Machine learning-based threshold prediction and classification.
 - Integration with modern IDE environments.
 - Advanced token categorization and analysis techniques.
-

11. Justification of Using C

Although Python with Pandas was initially considered, C was selected for the following reasons:

- Compiler systems are traditionally developed using C.
 - It provides greater low-level memory and system control.
 - It effectively demonstrates real-world lexical analysis and token processing.
-

12. Conclusion

This project successfully converts raw lexical data into a meaningful metric known as token density. It illustrates how compiler-level information can be utilized to evaluate code complexity, readability, and maintainability. The proposed approach is efficient, practical, and capable of being extended further through future enhancements and advanced analysis techniques.